

LAYER 4

DATA & ORACLE LAYER

Horizontal Decomposition Architecture

Self-Evolving & Self-Optimizing
Quadratic Voting DAO Governance Module

Agentic IDE Implementation Guide
Peer Module Architecture with Strict Interface Contracts

■■■ Technical Specification Document ■■■

TABLE OF CONTENTS

- 1. Executive Summary
- 2. Horizontal Architecture Overview
- 3. Module 1: Oracle Adapter Module
- 4. Module 2: On-Chain Indexer Module
- 5. Module 3: Off-Chain Fetcher Module
- 6. Module 4: Cryptographic Verifier Module
- 7. Module 5: Multi-Source Aggregator Module
- 8. Module 6: Data Normalization & Schema Engine
- 9. Module 7: Tamper-Proof Audit & Provenance Logger
- 10. Horizontal Interface Contract (Event Bus)
- 11. Recommended File Structure
- 12. Implementation Order
- 13. Critical Constraints Mapping
- 14. Interface Definitions Reference

1. EXECUTIVE SUMMARY

This document presents a horizontal decomposition of **Layer 4: Data & Oracle Layer** from the Self-Evolving & Self-Optimizing Quadratic Voting DAO Governance Architecture. The decomposition transforms the monolithic data layer into seven independent, peer-level modules that can be developed in parallel through an agentic coding IDE.

Each module is designed with strict interface contracts, cryptographic guarantees, and deterministic behavior to ensure the foundational principles of the DAO governance system remain intact: **Sybil resistance, optimization, and safety**.

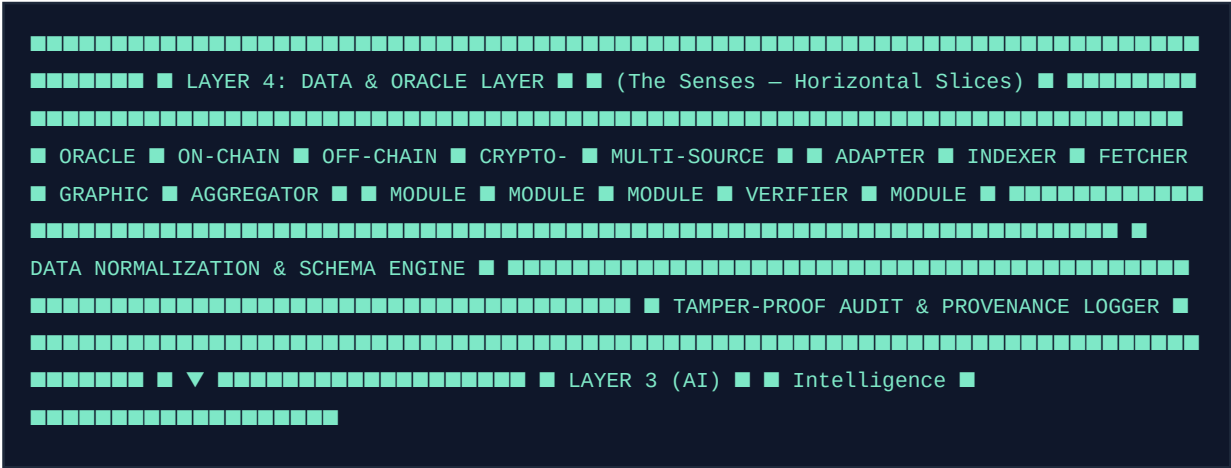
Key Design Principles:

- Absolute cryptographic verifiability of all data inputs
- Multi-source aggregation to prevent manipulation attacks
- Immutable audit trail with on-chain anchoring
- Modular independence enabling parallel agentic development
- Deterministic output schema for Layer 3 (AI/ML) consumption

2. HORIZONTAL ARCHITECTURE OVERVIEW

2.1 Layer 4 Structural Diagram

The horizontal decomposition splits Layer 4 into seven peer modules operating at the same architectural level, each with a single, well-defined responsibility. All modules communicate through a centralized event bus with strict type contracts.



2.2 Module Responsibilities Matrix

Module	Responsibility	Critical Constraint
Oracle Adapter	Abstract all oracle provider interactions	Fallback on provider failure
On-Chain Indexer	Extract & compute on-chain metrics	Block-height consistency
Off-Chain Fetcher	Collect quantifiable off-chain data	Source whitelist & rate limiting
Cryptographic Verifier	Verify cryptographic integrity	Every packet hashed & signed
Multi-Source Aggregator	Aggregate & reach consensus	Min 3 sources, outlier trimming
Schema Engine	Normalize to ML-consumable format	Versioned schema contracts
Audit Logger	Immutable provenance & forensics	Merkle root on-chain anchoring

3. MODULE 1: ORACLE ADAPTER MODULE

Responsibility: Abstract all oracle provider interactions (Chainlink, API3, Pyth, custom oracles).

3.1 Interface Contract

```
// interfaces/IOracleAdapter.ts interface IOracleAdapter { requestData(feedId: string,
params: OracleParams): Promise<OracleResponse>; verifyOracleSignature(response:
OracleResponse): boolean; getProviderName(): string; } // Concrete implementations: //
adapters/ChainlinkAdapter.ts // adapters/Api3Adapter.ts // adapters/PythAdapter.ts
```

3.2 Key Components

OracleRouter — Routes requests to the appropriate provider based on feed type and availability.

ResponseValidator — Checks oracle response format, timestamps, and staleness thresholds.

FallbackManager — Automatically switches to secondary oracle if primary fails or times out.

3.3 Failure Modes

- **Stale Data:** Reject if `block.timestamp - response.timestamp > MAX_STALENESS`
- **Invalid Signature:** Reject and log to Audit Logger, trigger fallback
- **Provider Outage:** Cascade to next provider in priority list
- **Consensus Mismatch:** Flag for Multi-Source Aggregator outlier detection

4. MODULE 2: ON-CHAIN INDEXER MODULE

Responsibility: Extract and compute on-chain metrics (voter turnout, token price, treasury velocity).

4.1 Interface Contract

```
// indexers/OnChainIndexer.ts interface OnChainMetrics { voterTurnout: bigint; // % of
eligible voters participating tokenPrice: bigint; // TWAP or oracle-reported
treasuryVelocity: bigint; // Inflow/outflow ratio over epoch proposalCount: number; //
Active/completed proposals } class OnChainIndexer { async snapshotMetrics(blockHeight:
number): Promise<OnChainMetrics>; async getVoterTurnout(proposalId: string):
Promise<bigint>; async getTreasuryVelocity(windowBlocks: number): Promise<bigint>; }
```

4.2 Key Components

EventScanner — Listens to governance contract events: VoteCast, ProposalCreated, TreasuryTransfer.

MetricCalculator — Computes derived metrics: turnout percentage, velocity ratios, quadratic distribution scores.

BlockHeightAnchor — Anchors all reads to a specific block for deterministic, reproducible snapshots.

4.3 Critical Invariants

- All metric reads must reference the same block height within a single snapshot
- Event logs must be verified against block header Merkle root
- Treasury velocity must exclude internal rebalancing transactions

5. MODULE 3: OFF-CHAIN FETCHER MODULE

Responsibility: Collect quantifiable off-chain data (sentiment, market data, social signals).

5.1 Interface Contract

```
// fetchers/OffChainFetcher.ts interface OffChainDataPoint { source: string; // e.g., "twitter_sentiment", "github_activity" timestamp: number; value: number; confidence: number; // 0.0 - 1.0 rawProof: Buffer; // Response hash/signature from source } class OffChainFetcher { async fetchSentiment(tokenSymbol: string): Promise<OffChainDataPoint>; async fetchMarketVolatility(): Promise<OffChainDataPoint>; async fetchDevActivity(repoUrl: string): Promise<OffChainDataPoint>; }
```

5.2 Key Components

HTTPSecureClient — Enforces TLS 1.3, certificate pinning, and timeout budgets.

SourceRegistry — Whitelist of approved off-chain endpoints with reputation scores and update frequencies.

RateLimitManager — Prevents fetcher from being throttled or banned by external APIs.

5.3 Security Considerations

- All off-chain sources must provide cryptographically signed responses
- Unknown sources are rejected at the SourceRegistry boundary
- Response hashes are immediately logged to the Audit Logger before processing
- Circuit breaker halts fetching if source error rate exceeds threshold

6. MODULE 4: CRYPTOGRAPHIC VERIFIER

MODULE

Responsibility: Guarantee every data input is cryptographically verifiable before it enters the aggregation pipeline.

6.1 Interface Contract

```
// crypto/CryptographicVerifier.ts interface VerificationResult { isValid: boolean;
dataHash: string; // keccak256 of normalized data signatureScheme: string; // "ECDSA",
"BLS", "ZK-Proof" verifierAddress: string; // On-chain verifier contract or public key }
class CryptographicVerifier { verifyMerkleProof(root: string, leaf: string, proof:
string[]): boolean; verifyZKProof(proof: ZKProof, publicInputs: any[]): boolean;
verifyECDSASignature(data: Buffer, sig: string, pubKey: string): boolean;
computeDataIntegrityHash(data: any): string; }
```

6.2 Key Components

SignatureValidator — Supports ECDSA, EdDSA, and BLS signature verification schemes.

MerkleProofChecker — Verifies batched oracle data (e.g., Chainlink Off-Chain Reporting consensus).

ZKProofVerifier — For zk-oracles or privacy-preserving attestations.

HashChainLogger — Chains hashes sequentially to detect insertion, deletion, or reordering attacks.

6.3 Verification Pipeline

1. Receive raw data packet from Oracle Adapter, Indexer, or Fetcher
2. Compute keccak256 hash of normalized data
3. Verify signature against known public key / verifier contract
4. If batched: verify Merkle inclusion proof
5. If ZK: verify proof against verification key
6. Emit VERIFICATION_PASSED or VERIFICATION_FAILED event to Event Bus

7. MODULE 5: MULTI-SOURCE AGGREGATOR MODULE

Responsibility: Prevent data-manipulation attacks by aggregating across multiple independent sources and reaching consensus.

7.1 Interface Contract

```
// aggregation/MultiSourceAggregator.ts interface AggregatedFeed { value: bigint; confidence: number; sourceCount: number; outlierSources: string[]; // Sources that were discarded aggregationMethod: "median" | "weighted_mean" | "stdev_trim"; } class MultiSourceAggregator { aggregate( sources: DataSource[], method: AggregationMethod, minSources: number ): Promise<AggregatedFeed>; detectOutliers(sources: DataSource[], threshold: number): string[]; computeConsensusDeviation(feed: AggregatedFeed): number; }
```

7.2 Key Components

- MedianAggregator** — Default strategy; robust against single-source manipulation.
- StdevTrimAggregator** — Removes outliers beyond N standard deviations from the mean.
- WeightedAggregator** — Weights by source reputation score and historical accuracy.
- ConsensusQuorum** — Fails the entire aggregation if fewer than minSources agree within tolerance.
- DeviationAlert** — Triggers Layer 5 veto alert if consensus deviation exceeds anomaly threshold.

7.3 Aggregation Strategies Comparison

Strategy	Use Case	Manipulation Resistance	Latency
Median	Default for all feeds	High (needs 51% control)	Low
Stdev Trim	Noisy data environments	Very High	Medium
Weighted Mean	Known reputable sources	Medium-High	Low
Consensus Quorum	High-stakes decisions	Maximum	High

8. MODULE 6: DATA NORMALIZATION & SCHEMA ENGINE

Responsibility: Convert raw, heterogeneous data into a standardized, typed schema consumable by Layer 3 (AI/ML).

8.1 Interface Contract

```
// schema/IntelligenceLayerSchema.ts interface IntelligenceInput { epoch: number;
timestamp: number; governance: { voterTurnout: NormalizedMetric;
quadraticDistributionScore: NormalizedMetric; proposalSuccessRate: NormalizedMetric; };
economic: { tokenPrice: NormalizedMetric; treasuryVelocity: NormalizedMetric;
volatilityIndex: NormalizedMetric; }; social: { sentimentScore: NormalizedMetric;
devActivity: NormalizedMetric; }; provenance: DataProvenance; // Audit trail from Logger }
class SchemaEngine { normalize(rawData: RawAggregatedData): IntelligenceInput;
validateSchema(input: IntelligenceInput): boolean; serializeForML(input:
IntelligenceInput): Buffer; // ONNX-compatible tensor }
```

8.2 Key Components

TypeNormalizer — Converts all values to fixed-point decimals or normalized floats (0-1 scale).

SchemaValidator — JSON Schema / Zod validation ensuring all required fields are present and typed correctly.

TensorSerializer — Formats output as ONNX-compatible tensors or NumPy arrays for ML pipeline consumption.

VersionRegistry — Schema versioning to prevent breaking Layer 3 on format updates.

8.3 Normalization Rules

- All percentages: 0.0 to 1.0 (float64)
- All token amounts: wei (uint256) or normalized by total supply
- Timestamps: Unix epoch milliseconds (int64)
- Confidence scores: 0.0 to 1.0, derived from source count and deviation
- Missing data: represented as NaN with confidence = 0.0

9. MODULE 7: TAMPER-PROOF AUDIT & PROVENANCE LOGGER

Responsibility: Immutable audit trail of every data operation for forensic analysis and Layer 5 veto investigation.

9.1 Interface Contract

```
// audit/ProvenanceLogger.ts interface ProvenanceRecord { recordId: string; timestamp: number; operation: "FETCH" | "VERIFY" | "AGGREGATE" | "NORMALIZE"; inputHashes: string[]; outputHash: string; operatorModule: string; // Which component performed the operation blockAnchor?: number; // On-chain anchor if applicable } class ProvenanceLogger { append(record: ProvenanceRecord): void; getMerkleRoot(): string; anchorToChain(root: string): Promise<string>; // Returns tx hash verifyIntegrity(recordId: string): boolean; }
```

9.2 Key Components

AppendOnlyLog — Write-once, WORM storage (local + IPFS/Filecoin for redundancy).

MerkleTreeBuilder — Builds Merkle root of all records per epoch for compact verification.

ChainAnchor — Posts Merkle root to a cost-efficient L2 or Ethereum calldata for immutability.

QueryAPI — Allows Layer 5 (Veto Layer) to inspect full provenance of any data point.

9.3 Audit Lifecycle

1. Every module operation generates a ProvenanceRecord with input/output hashes
2. Records are appended to the local WORM log
3. At epoch boundary, Merkle tree is built from all records
4. Merkle root is anchored on-chain via low-cost transaction
5. Layer 5 can query any record by ID and verify against Merkle root

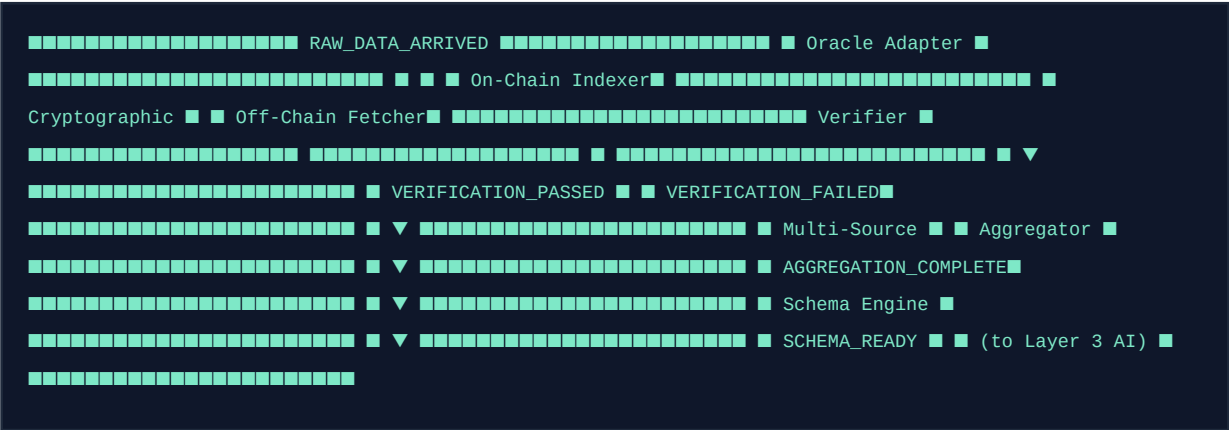
10. HORIZONTAL INTERFACE CONTRACT (EVENT BUS)

All modules communicate through a typed event bus with strict contracts. This is the glue that enables parallel development.

10.1 Event Type Definitions

```
// bus/Layer4EventBus.ts
type Layer4Event = | { type: "RAW_DATA_ARRIVED"; payload: RawDataPacket; source: string } | { type: "VERIFICATION_PASSED"; payload: VerifiedData; hash: string } | { type: "VERIFICATION_FAILED"; payload: RejectedData; reason: string } | { type: "AGGREGATION_COMPLETE"; payload: AggregatedFeed; method: string } | { type: "SCHEMA_READY"; payload: IntelligenceInput; epoch: number } | { type: "AUDIT_ANCHORED"; payload: string; // Merkle root txHash: string };
```

10.2 Event Flow Diagram



11. RECOMMENDED FILE STRUCTURE

Organized for agentic IDE parallel development with clear module boundaries.

```
/src/layer4/ ■■■ index.ts # Layer 4 bootstrap & wiring ■■■ config/ ■ ■■■ oracle.config.ts
# Provider endpoints, fallback rules ■ ■■■ sources.config.ts # Whitelisted off-chain
sources ■ ■■■ aggregation.config.ts # Min sources, deviation thresholds ■■■ interfaces/ ■
■■■ oracle.adapter.interface.ts ■ ■■■ data.schema.interface.ts ■ ■■■ audit.interface.ts
■■■ oracle-adapters/ ■ ■■■ chainlink.adapter.ts ■ ■■■ api3.adapter.ts ■ ■■■
pyth.adapter.ts ■ ■■■ oracle.router.ts ■■■ indexers/ ■ ■■■ on-chain.indexer.ts ■ ■■■
event.scanner.ts ■ ■■■ metric.calculator.ts ■■■ fetchers/ ■ ■■■ off-chain.fetcher.ts ■
■■■ https.client.ts ■ ■■■ source.registry.ts ■■■ crypto/ ■ ■■■ verifier.ts ■ ■■■
signature.validator.ts ■ ■■■ zk-proof.verifier.ts ■■■ aggregation/ ■ ■■■
multi-source.aggregator.ts ■ ■■■ median.strategy.ts ■ ■■■ stdev-trim.strategy.ts ■ ■■■
outlier.detector.ts ■■■ schema/ ■ ■■■ normalization.engine.ts ■ ■■■
intelligence.schema.ts ■ ■■■ tensor.serializer.ts ■■■ audit/ ■ ■■■ provenance.logger.ts
■■■ merkle.tree.ts ■ ■■■ chain.anchor.ts ■■■ bus/ ■■■ layer4.event-bus.ts
```

12. IMPLEMENTATION ORDER

Optimized for agentic coding with parallel development tracks.

Phase 1: Foundation (Sequential)

1. **interfaces/** and **bus/** — Establish contracts first. All downstream modules depend on these.
2. **config/** — Centralized configuration for all environment-specific values.

Phase 2: Parallel Development Tracks

Track A: Safety & Trust (Foundational)

- **audit/** — Provenance logger, Merkle tree, chain anchor
- **crypto/** — Signature validator, ZK proof verifier

Track B: On-Chain Data Pipeline

- **oracle-adapters/** — Chainlink, API3, Pyth adapters + router
- **indexers/** — Event scanner, metric calculator

Track C: Off-Chain Data Pipeline

- **fetchers/** — HTTPS client, source registry, rate limiter

Phase 3: Integration (Sequential)

3. **aggregation/** — Depends on A, B, C tracks completing their interfaces
4. **schema/** — Depends on aggregation output format

Phase 4: Wiring & Bootstrap

5. **index.ts** — Bootstrap with dependency injection and event bus wiring
6. **Integration tests** — End-to-end pipeline validation

13. CRITICAL CONSTRAINTS MAPPING

Mapping the original DAO governance document constraints to Layer 4 implementation.

Document Constraint	Implementation Module	Specific Mechanism
Cryptographically verifiable	Cryptographic Verifier	keccak256 + ECDSA/BLS/ZK on every packet
Multi-source aggregation	Multi-Source Aggregator	minSources >= 3, median default, outlier trimming
Tamper-proof data	Audit & Provenance Logger	WORM log + Merkle tree + on-chain anchoring
Prevent data-manipulation	Multi-Source Aggregator + Event Bus	ConsensusQuorum + DeviationAlert → Layer 5 veto
No direct treasury access	All modules	Zero treasury interaction in Layer 4
Hardcoded parameter bounds	Config layer	Strict upper/lower bounds in aggregation.config.ts
Deterministic execution	On-Chain Indexer	BlockHeightAnchor for reproducible snapshots

13.1 Safety Principle Enforcement

The **Safety Principle** from the original architecture states: *"Any automated parameter adjustment must be mathematically bounded to prevent runaway feedback loops or malicious capture."*

In Layer 4, this translates to:

- All aggregation parameters (minSources, deviationThreshold) are hardcoded in config
- The Multi-Source Aggregator cannot be instructed to reduce minSources below 3
- The Cryptographic Verifier cannot be bypassed by any module — it is mandatory
- The Audit Logger records every parameter change attempt for Layer 5 review

14. INTERFACE DEFINITIONS REFERENCE

Complete type definitions for all cross-module interfaces.

14.1 Oracle Adapter Interfaces

```
interface OracleParams { feedId: string; maxStaleness: number; // seconds fallbackProviders:
string[]; timeoutMs: number; } interface OracleResponse { value: bigint; timestamp: number;
signature: string; provider: string; blockNumber?: number; }
```

14.2 Data Source Interfaces

```
interface DataSource { id: string; type: "on-chain" | "off-chain" | "oracle"; value: bigint;
timestamp: number; confidence: number; proof: VerificationResult; metadata: Record<string,
any>; } interface RawDataPacket { sourceId: string; rawPayload: Buffer; receivedAt: number;
transportProtocol: "https" | "websocket" | "rpc"; } interface VerifiedData { source:
DataSource; integrityHash: string; verifiedAt: number; verifierModule: string; }
```

14.3 Aggregation Interfaces

```
type AggregationMethod = "median" | "weighted_mean" | "stdev_trim"; interface
AggregationConfig { method: AggregationMethod; minSources: number; // minimum: 3
maxDeviation: number; // maximum allowed std dev outlierThreshold: number; // z-score
threshold reputationWeights?: Map<string, number>; } interface RejectedData { sourceId:
string; reason: "VERIFICATION_FAILED" | "OUTLIER_DETECTED" | "STALE" | "SOURCE_UNTRUSTED";
originalValue: bigint; expectedRange?: [number, number]; }
```

14.4 Normalized Metric Type

```
interface NormalizedMetric { value: number; // 0.0 to 1.0 for ratios, raw wei for amounts
confidence: number; // 0.0 to 1.0 sourceCount: number; timestamp: number; rawValue?: bigint;
// Original unnormalized value } interface DataProvenance { recordIds: string[]; merkleRoot:
string; anchoredAt?: number; // Block number of on-chain anchor anchorTxHash?: string; }
```


APPENDIX A: GLOSSARY

QV

Quadratic Voting — A voting mechanism where $\text{cost} = \sum (v_i^2)$, allowing preference intensity expression.

ZK-Proof

Zero-Knowledge Proof — Cryptographic proof that verifies a statement without revealing underlying data.

WORM

Write Once Read Many — Storage that prevents modification after initial write.

TWAP

Time-Weighted Average Price — Price averaged over a time window to resist manipulation.

OCR

Off-Chain Reporting — Chainlink's consensus mechanism for oracle networks.

Merkle Tree

Cryptographic tree structure enabling efficient verification of data integrity.

OODA Loop

Observe-Orient-Decide-Act — Decision cycle framework adapted for autonomous systems.

Agentic IDE

Integrated Development Environment with AI agents capable of autonomous code generation.

Sybil Attack

Identity spoofing attack where one entity creates multiple fake identities.

Ragequit

DAO mechanism allowing members to exit with proportional share of treasury.

■■■■ END OF DOCUMENT ■■■■